

Database Constraints

A constraint is a database rule that enforces data integrity by restricting the values that can be inserted, updated, or deleted.

Example With Constraints

```
CREATE TABLE Students(  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Age INT CHECK(Age>=0)  
);
```

Business Rules vs Database Rules

Business Rule	Database Rule
Every employee must have a salary.	Salary NOT NULL
Email should be unique.	Email UNIQUE
Age must be positive.	CHECK(Age>0)

Column-Level Constraints

Applied directly to one column.

Example

```
CREATE TABLE Students(  
    Name VARCHAR(100) NOT NULL  
);
```

Table-Level Constraints

Defined after all columns.

Example

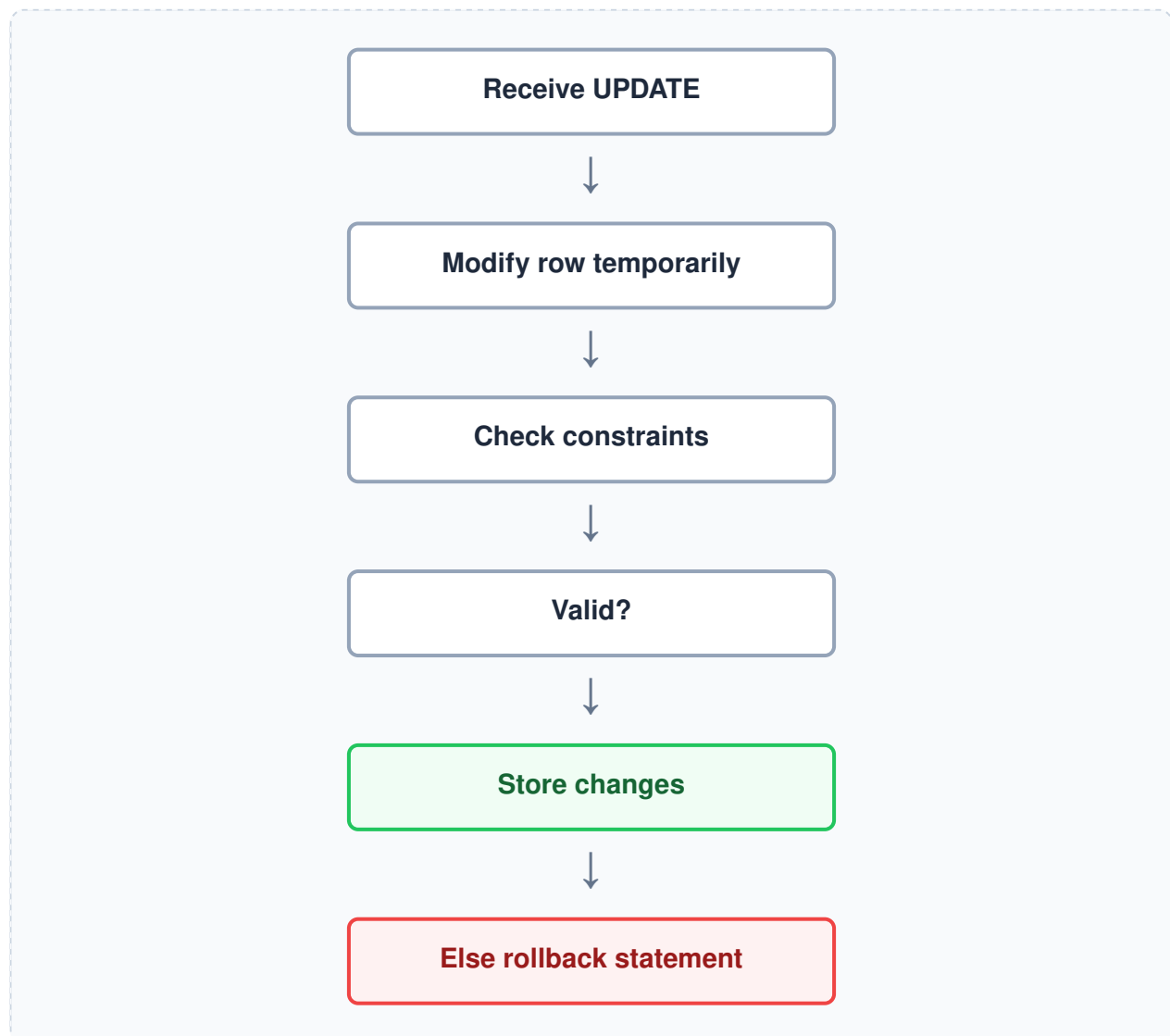
```
CREATE TABLE Students(  
    StudentID INT,  
    CourseID INT,  
  
    PRIMARY KEY(StudentID, CourseID)  
);
```

During UPDATE

Suppose

```
UPDATE Students SET Age=-5 WHERE StudentID=1;
```

Flow



Modifying Constraints

Constraints don't have to exist from day one.
They can be created, modified, or removed over time

```
ALTER TABLE Students ADD CONSTRAINT uq_email UNIQUE(Email);
```

Dropping Constraints

Suppose you no longer need a constraint.

```
ALTER TABLE Students DROP INDEX uq_email;
```

Dropping Constraints

Suppose you no longer need a constraint.

```
ALTER TABLE Students DROP INDEX uq_email;
```

Renaming Constraints

Unlike SQL Server or PostgreSQL, MySQL does not provide a general RENAME CONSTRAINT statement.

Typical approach:

- Drop the old constraint.
- Recreate it with the new name.

Some constraint types (like indexes backing UNIQUE) can be renamed separately using index-related commands.